

1. What is Spring Boot and why is it used?

Spring Boot is a framework developed on top of the Spring Framework that simplifies the development of production-grade Spring applications. It reduces lots of development time and increases the productivity of developers by doing a lot of auto-configuration, starter dependency and embedded server (Tomcat, Jetty, and Undertow).

Why used:

- Minimal configuration and fast setup.
- Support for production-ready capabilities like metrics, security and monitoring.
- Less Complex deployment (runs as a standalone JAR).
- Microservice Friendly.

2. Difference between Spring Framework and Spring Boot:

Feature	Spring Framework	Spring Boot
Configuration	Manual (XML or Java-based)	Auto-configuration
Server Setup	Requires external server	Comes with embedded Tomcat/Jetty
Dependency Management	Must add individually	Uses starter dependencies
Goal	Provides foundation for enterprise apps	Simplifies Spring app creation
Deployment	Deploy as WAR	Run as standalone JAR

Example:

In Spring, you configure beans manually, but in Spring Boot, `@SpringBootApplication` handles component scanning, configuration, and auto-setup.

3. Inversion of Control (IoC) and Dependency Injection (DI):

- Inversion of Control (IoC): It means the decoupling the creation and running of objects in your application.
- Dependency Injection (DI): Spring provides dependency of classes through DI.

Example:

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Here, Spring injects an instance of PaymentService automatically — that's DI in action.

4. Purpose of application.properties / application.yml:

- These files also externalize configurations.
- Therefore, code does not need to be recompiled for an environmental change.

Examples:

```
- application.properties: properties    server.port=8081
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root    spring.datasource.password=1234 -
application.yml: yaml    server:    port: 8081    spring:    datasource:
url: jdbc:mysql://localhost:3306/mydb    username: root    password:
1234
```

5. What is a REST API and HTTP methods used:

- REST API (Representational State Transfer): This is an architectural style which can be used for systems to communicate over standard operations on HTTP.

HTTP Methods: - **GET:** Retrieve resource. - **POST:** Create a new resource. - **PUT:** Update an existing resource. - **PATCH:** Partially update resource. - **DELETE:** Remove resource.

Example:

```
@RestController
@RequestMapping("/users")
public class UserController {
    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) { ... }
}
```

6. Spring Data JPA, Entity, and Repository:

- **Spring Data JPA:** A module that simplifies data access using the JPA standard. It handles CRUD operations automatically through repositories.
- **Entity:** A POJO mapped to a database table using `@Entity`.

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
}
```

- **Repository:** An interface that provides CRUD operations.

```
public interface UserRepository extends JpaRepository<User, Long> {}
```

Spring automatically generates the implementation.

7. @Component, @Service, @Repository, @Controller, @RestController:

Annotation	Layer	Purpose
@Component	Generic	Marks a Spring-managed bean
@Service	Service	Contains business logic
@Repository	Data Access	Handles persistence and database exceptions
@Controller	Web (MVC)	Handles web requests and returns views
@RestController	REST	Returns JSON/XML responses (combines @Controller + @ResponseBody)

Example:

```
@Service
public class ProductService {}
```

Spring automatically detects it through component scanning.

8. @Autowired:

Spring Boot's @Autowired annotation plays a significant role for making the Application tightly coupled between objects, which is very important when we are working on large projects and provides flexibility to our code. In this article, we'll explore the @Autowired annotation and discuss about its significance and some best practices for Spring Boot applications. How @Autowired helps development and maintain ability of applications will be clearly illustrated.

Example:

```
@Autowired
private UserRepository userRepository;
```

When to avoid: - For testability, use constructor injection instead of field injection. Steer clear of utilizing it in static fields. Avoid using @Autowired (required = false) or @Qualifier for multiple beans if the dependency is optional.

9. Exception Handling in Spring Boot (@ControllerAdvice):

- For controllers, @ControllerAdvice offers global exception handling.

Example:

```
@ControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<String> handleNotFound(ResourceNotFoundException
ex) {
        return
ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
}
```

It minimizes redundant try-catch blocks and guarantees consistent error answers.

10. Role of Maven/Gradle in a Spring Boot project:

Both are **build automation and dependency management tools**.

- Handle downloading of JAR files (dependencies) from repositories.
- Automate build, testing, and packaging.
- Can run Spring Boot applications directly.

Example: - Maven: Uses pom.xml bash `mvn spring-boot:run` - **Gradle:** Uses build.gradle bash `gradle bootRun` They simplify dependency handling and ensure project consistency across environments.